M6：终端界面与端到端集成

编辑器、视口、差分绘制和生命周期

M6：终端界面与端到端集成
- 背景、目标与范围
- 行为规格
  - M6-S01：启动与退出
  - M6-S02：屏幕区域与视口
  - M6-S03：编辑器缓冲和光标
  - M6-S04：键盘分帧与粘贴
  - M6-S05：提交、中断与命令路由
  - M6-S06：字符宽度与安全文本
  - M6-S07：差分绘制
  - M6-S08：状态和 Git 页脚
  - M6-S09：真实模型接入与验收边界
- 数据结构与端口
- 详细设计与模式
  - 表示模型与单向数据流
  - 编辑器与布局分离
  - 差分缓存与生命周期
  - 临时视图与已提交消息
- 验收矩阵
- 布局与交互推演
  - grapheme和显示宽度
  - 字符串缩短和清行
  - 活动输入不丢失
  - 端到端验收脚本
- 备选方案、风险与最终交付

M6：终端界面与端到端集成

属性	内容
状态	设计提案 1.0
前置	M1至 M5
交付	`@local/tui`、默认交互入口、一种真实模型适配
验收	M6-A01 至 M6-A12

背景、目标与范围

交互界面同时接收键盘输入、模型流和工具状态，不能让屏幕刷新控制领域执行顺序。终端还会处理 ANSI 控制序列，未经处理的文件/模型文字可能移动光标或伪造界面。

目标是一个可恢复终端状态、支持中文基本输入与多行编辑的界面；界面状态由领域事件单向派生。非目标是完整 Markdown 渲染、图片、鼠标、终端模拟器、所有 Unicode 终端差异和无限滚动历史。基线测试的宽度规则固定；真实终端差异作为兼容性限制说明。

行为规格

M6-S01：启动与退出

仅 stdin/stdout 均为 TTY 时进入交互，否则退出 2，提示使用 print/rpc。启动前保存 raw mode 等终端状态，进入 alternate screen，启用 bracketed paste，隐藏光标后再建立输入监听器。初始化失败也走 finally，不能让 shell 留在 raw mode。

正常退出、SIGINT、异常、模型失败和终端写失败统一调用幂等 cleanup：停止读取输入→取消活动任务→等 Session/工具清理→停止计时器→关闭 MCP/Session→禁用 paste→显示光标→离开 alternate screen→恢复原 raw mode。错误诊断在终端恢复后写 stderr。

外部 SIGINT 请求退出；键盘 Ctrl+C 由 raw mode 输入解析，行为见 S05，不能重复触发两套处理。

M6-S02：屏幕区域与视口

屏幕按从上到下显示 transcript 区、状态区、编辑器区。transcript 高度=rows-状态行数-编辑器可见行数，至少 1；不足时进入紧凑模式，仅显示状态和输入最后一行，不能输出负坐标。终端宽高最小可接受 10×3，更小仅输出经过裁剪的 `Terminal too small` 并等 resize，不启动新输入任务。

transcript 每条 user/assistant/tool result 是显示块，按 M1 提交顺序；当前流式 assistant 作为暂存块，message_end 原位替换，不重复追加。工具 execution_update 显示状态预览，真正 transcript 内容以 tool result message_end 为准。

只保留最近 10000 条显示逻辑行，超出从头移除，状态提示 `older output in session log`。这不删除 Session 历史。PageUp/PageDown 每次移动一页减一行，Home/End 属于编辑器，不改变 transcript。用户在底部时新输出自动跟随；已向上滚动时保持视口，并显示未读行数；滚回底部清零。

M6-S03：编辑器缓冲和光标

缓冲以 grapheme cluster 为单位，使用 Intl.Segmenter 分段；cursor 是 cluster 索引，不能以 UTF-16 code unit 删除半个 emoji。text 是真实内容，soft wrap 只影响显示，不插入换行。显式 LF 视为一个换行 cluster。

Left/Right 移动一个 cluster；Backspace 删除前一个，Delete 删除后一个；Home/End 移到当前逻辑行首/尾。Up/Down 在非首/末显示行按显示列移动，保留 preferredColumn；在首个显示行向上可进入输入历史，在末个显示行向下可返回下一条历史/草稿。历史最多100条被接受的模型任务原始输入（包括显式技能命令），不存空输入、管理命令或失败 slash command，不跨会话持久化。

输入最大 64 KiB UTF-8。普通字符超过上限拒绝本次插入并显示 LIMIT；paste 超限整次拒绝，不截成半条任务。控制字符除 LF/TAB 之外不进入正文；TAB 在输入时变成4个空格。

M6-S04：键盘分帧与粘贴

终端 chunk 不是一个按键。输入解析器累积 ANSI 前缀，识别方向键、Delete、PageUp/PageDown 与 bracketed paste 开始/结束。转义序列可跨 chunk；单个 ESC 用30ms fake clock 可测的等待窗口，未形成已知序列则按 Esc 操作。未知 CSI 忽略并诊断，不把原码写入用户消息。

bracketed paste 从 `ESC[200~` 到 `ESC[201~`，内部 CRLF 规范 LF，不把回车当提交，粘贴内容整体插入。粘贴中的 ESC 不触发 abort。EOF 或超过64 KiB但没收到结束标记时取消本次 paste 并诊断，不提交任务。

终端常不能区分 Shift+Enter 与 Enter；基线用 Ctrl+J 插入 LF，Enter 提交。可额外支持 CSI-u 键盘协议，但不将其列为所有终端都能工作的前提。

M6-S05：提交、中断与命令路由

输入	idle	running/compacting
Enter	非空输入提交；成功接受后清缓冲	保留缓冲，提示使用 Ctrl+S/Ctrl+F 入队
Ctrl+J	插入 LF	插入 LF
Ctrl+S	普通提交	非空缓冲送 steer，成功后清空
Ctrl+F	普通提交	非空缓冲送 follow-up，成功后清空
Esc	清除状态提示，不退出	请求取消当前任务，输入不清
Ctrl+C	输入非空先清空；空输入退出	请求取消，等终态后再按才退出
Ctrl+D	输入为空退出；非空无操作	请求取消并在任务结束后退出

输入非空指 trim 后非空，但提交保留原文。API 拒绝时保留缓冲并显示 code，不丢用户输入。队列成功仅代表入队，UI 显示数目；恢复或关闭时未消费项不保存。

slash commands 只在 idle 路由：`/reload`、`/compact [instructions]`、`/skill:<name> [task]`、`/branches`、`/branch <entryId>`、`/branch root`、`/quit`。未知以 `/` 开头的输入返回 INVALID_INPUT，不悄悄发给模型；要发送字面 slash 用 `//`，删除第一个 `/` 后作为普通任务。`/branches` 用 Session.listBranches 显示可复制的完整 entryId、预览和当前标记，不调用模型。`/branch` 只切会话，不切工作树，成功后重建显示投影并提示文件未回滚。

M6-S06：字符宽度与安全文本

领域文字输出前必须 sanitize。丢弃 ESC 开始的 ANSI 序列、OSC/DCS 和 C0/C1 控制字符，保留 LF；CR 规范为 LF，TAB 按4列展开。不允许模型/文件文本输出 BEL、光标移动、屏幕清除或 OSC52 剪贴板操作。渲染器自身产生的受信 ANSI 与不受信文字分通道，不能先拼接再粗暴 strip。

宽度基线：组合符0列，ASCII普通字符1列，东亚宽字符和 emoji cluster2列。使用已固定版本字符宽度库并补 grapheme 组合，不能用 string.length。宽度不足2列时宽字符单独放下一行；终端总宽度1的特殊态由 S02 小屏提示处理。

自动换行避免写最后一列触发终端滚动：逻辑布局使用 `columns-1` 可用列。绘制前截断所有行到该宽度，光标坐标按显示列计。Unicode ambiguous width 按1列；某些终端配置不同，需要用户将终端设为相同规则，不宣称所有字体精确一致。

M6-S07：差分绘制

`render(view,columns,rows)` 纯函数生成 frame `{lines,cursor}`，lines 为清理后的纯文本，每行不超可用宽度、总行数不超 rows。Renderer 保存 previousFrame，对比行字符串，只重写变化行；旧行比新行长时必须清到行尾；新帧更短时必须清掉剩余旧行。

resize 使布局和 previousFrame 全失效，下一次全绘。首次绘制全绘。普通流式 update 合并到最多30fps，message_end、错误、取消和退出前强制 flush。不得丢最终消息。多次 render 请求通过 dirty 标志和单个调度任务合并，避免重入光标操作。

不要求最少 ANSI 字节，只要求可观察屏幕正确且同帧重复绘制不重写内容。界面格式样式可以自行设计，但三个区域和上述输入语义必须一致。

M6-S08：状态和 Git 页脚

状态含 phase、provider/model、workspace/cwd、branch/detached/unavailable、当前 turn 数、工具数量和两队列长度。取消请求后显示 cancelling，直到终态；不能提前显示 idle。

Git 元数据最多每500ms查询一次，合并重复刷新；挂起查询结束后才发下次。错误只改变 branchState，不影响模型任务。退出前取消刷新 timer/子进程。页脚不把秘密环境变量或工具完整参数渲染出来。

M6-S09：真实模型接入与验收边界

最终 CLI 必须有一种真实供应商 ModelAdapter。适配器负责 HTTP/SDK、供应商工具格式转换、token usage、取消和错误归一化。先聚合完整工具参数，再发 M1 final；不把供应商 stream chunk 原样泄漏给 Core。

实现者在 README 写清 provider、模型配置变量、密钥变量名、运行命令、支持的 stop reason 和已知限制。无密钥返回 INVALID_CONFIG，除非用户显式 –demo；不自动切假模型。真实 smoke test 用只读临时任务且显式启用，日常测试默认不联网。

数据结构与端口

```ts
interface Terminal {
  readonly columns: number;
  readonly rows: number;
  write(text: string): Promise<void>;
  setRawMode(enabled: boolean): void;
  subscribeInput(fn: (bytes: Uint8Array) => void): () => void;
  subscribeResize(fn: (columns: number, rows: number) => void): () => void;
  close(): Promise<void>;
}
type EditorState = {
  clusters: string[];
  cursor: number;
  preferredColumn: number | null;
  history: string[];
  historyIndex: number | null;
  draft: string;
};
type DisplayBlock = {
  id: string;
  kind: "user" | "assistant" | "tool" | "notice";
  text: string;
  pending: boolean;
  isError: boolean;
};
type ViewModel = {
  blocks: DisplayBlock[];
  editor: EditorState;
  status: { phase: string; label: string; error?: string };
  scrollOffset: number;
  unreadLines: number;
};
type Frame = {
  lines: string[];
  cursor: { row: number; column: number; visible: boolean };
};
```

tui 不 import Core。CLI adapter 把 AgentEvent 转成 DisplayAction，例如 beginAssistant/updateAssistant/commitMessage/setPhase，并将 KeyAction 转成 Session API。Terminal 是真实进程与 FakeTerminal 的共同端口；FakeTerminal 应维护屏幕单元或至少记录 ANSI 操作并解释清行，不能只做 write 次数计数。

详细设计与模式

表示模型与单向数据流

领域事件 → CLI mapper → view reducer → frame layout → terminal diff，是唯一数据方向。键盘事件先转成命令，再调用 Session，领域结果回到同一 reducer。UI 不直接 push Core.messages，也不把屏幕文本当恢复存档。这是 Presentation Model 与单向数据流；不要求使用 React 或状态管理框架。

编辑器与布局分离

Editor reducer 处理 grapheme 插入/删除/历史，与终端宽度无关；layout 负责 wrap 和光标映射。维护 clusterIndex→显示行列的映射表，让左右与上下移动可测试。长度 n 的全量布局 O(n)，最大64KiB输入可接受；不必一开始实现 rope。若输入预算扩大再比较 piece table/rope，而不是为设计模式名称引入复杂结构。

差分缓存与生命周期

Renderer 只有 previousFrame、dirty、scheduled、closed 四类状态。布局后逐行比较 O(rows×columns)，重写 d 个变化行 O(d×columns)。所有写经一个 Promise 链串行，resize 不在半帧中插入另一套 cursor 指令；完成当前写后全量重绘。

清理使用一次性 disposer 列表，按注册逆序执行；一个 disposer 抛错仍继续后续恢复。raw mode 原状态必须保存，不无条件 setRawMode(false) 假设启动前状态。退出恢复是 finally 的职责，不散落在每个按键处理器。

临时视图与已提交消息

message_start 创建临时 assistant block，update 改其文本，end 替换为最终消息。取消的部分文字也会有 end，所以 UI 不自己猜“最后 chunk 是不是 final”。工具预览放状态区，正式结果按领域 message_end 顺序插入 transcript，避免并行工具造成显示历史和日志次序不同。

验收矩阵

编号	操作	断言
M6-A01	非TTY、启动异常、正常退出	退出码/提示正确；终端恢复一次
M6-A02	两帧只有一行变化；第二帧行更短	只需重写变化行并清尾；残字消失
M6-A03	frame 行数减少、resize	清旧行；resize 全重绘；无越界光标
M6-A04	输入中文、组合重音、家庭emoji再Backspace	删除一个 grapheme；无半代理或残组合符
M6-A05	多行导航、Home/End、历史草稿	光标按显示列；草稿可恢复；软换行不改正文
M6-A06	按键序列和UTF-8跨chunk；paste多行	解析一条操作；paste不提交；64KiB上限整次拒绝
M6-A07	idle/running 的 Enter/Ctrl+S/Ctrl+F	按状态提交或排队；失败缓冲保留
M6-A08	Ctrl+C/Esc/Ctrl+D、外部SIGINT	取消与退出语义一致；无重复清理
M6-A09	模型文本含ANSI/OSC52/BEL	无终端副作用；仅受信渲染器发控制码
M6-A10	大量updates、滚动、窗口极小	<=30fps；最终状态不丢；视口与未读数正确

编号	操作	断言
M6-A11	read→edit→read 的真实Session+假模型	屏幕最终文本、文件和日志一致；重开无重复工具
M6-A12	真实适配器的录制/人工构造供应商流	转换符合M1；无key明确失败；demo显式可用

真实终端手工检查一次中文宽度、多行paste、取消和退出后 shell 回显。自动检查用 FakeTerminal 断言最终单元屏幕而非只匹配 ANSI 字符串；录屏不是正确性测试。

布局与交互推演

grapheme和显示宽度

缓冲a中é□含4个grapheme；显示宽度为1+2+1+2=6列。cursor=4时Backspace删除整个□再Left越过整个é，不把重音留在上一字符上。布局不能用JavaScript length推断列，否则中文和emoji后的光标都会错位。

当可用宽度=5，前三个grapheme占4列，最后□移到下一显示行；text仍是一行，不插入LF。Home回逻辑行首，不是当前soft-wrap行首；Up/Down按显示行移动，这是编辑器和layout需要共享映射表的原因。规格中“当前逻辑行”的Home/End与“按显示列”的Up/Down不可混为一谈。

字符串缩短和清行

旧屏幕某行是`tool: searching...`，新行是`done`。若只写done，屏幕残留`done: searching...`。正确差分先定位、写新文本、清至行尾。新帧少了两行时也要逐行清旧尾，不能只更新currentLines数组。

同帧重复render可以移动/显示光标，但不重写正文。resize即使文本未变，也须重新wrap并全绘，因为旧行比较已不成立。渲染器不应解析模型Markdown作为ANSI命令执行；纯文本是基线。

活动输入不丢失

模型正在调用工具，用户输入“改用另一文件”后Enter：缓冲保留并提示入队方法，不调用第二个prompt。Ctrl+S成功后缓冲清空、steer计数+1；直到M1消费边界才出现在transcript。取消后未消费steer仍留队列，clear_queue可显示并取回，不自动丢弃。

`/skill:review`找不到时保留原输入以便修正；`/reload`成功不加输入历史；普通任务被Session接受后即加入历史，即使后来模型失败。这区分“提交失败”和“运行失败”。

端到端验收脚本

1. 临时工作区创建a.txt=`hello`，启动–demo交互与真实Session。
2. 输入“改为world”；脚本模型依次返回read、edit、read、最终文本。

3. 断言磁盘a.txt=`world`；日志有3个call和3个配对result，顺序稳定。
4. 在FakeTerminal中断言最终文本和idle状态，输入区为空。
5. close后重新open同一会话；历史仍在，工具执行计数仍为3，无恢复重放。
6. 新输入前刷新资源，context变化只追加一次；关闭后锁已释放。

这条验收横跨模块边界，不替代各模块故障测试。真实供应商smoke只确认适配器能连通，不要求模型恰好选择相同工具或文案。

备选方案、风险与最终交付

使用现成大型 TUI 框架可以减少终端兼容成本，但本项目要练习编辑器、渲染状态和领域边界，选最小自研接口。每帧全屏清除最容易实现，却有闪烁和滚动问题，因此在全绘基线通过后增量做逐行差分。

先做 FakeTerminal 和静态 frame，再做纯编辑器、输入解析、差分绘制、领域 mapper，最后接真实终端和供应商。最终提交 README、配置示例、里程碑验收命令和限制说明；确保项目默认不需要密钥运行 –demo，同时真实模式不会伪装成功。